AMT Benchmark Application Specification: Gravitational N-Body Simulation with Barnes-Hut v1.0

Gregor Daiss

May 2026

Executive Summary

In this whitepaper, we outline the plan of using the Barnes-Hut algorithm for gravitational problems as a benchmark problem to test and compare Asynchronous Many-Task (AMT) Runtime Systems with each other and with traditional MPI implementations.

The Barnes-Hut algorithm is both straight-forward as a concept, yet challenging when it comes to creating distributed implementations. This combination of easy-to-grasp but hard-to-implement, makes it an ideal benchmark to showcase features of AMT systems meant to ease the creation of distributed applications. The dynamic nature of the problem also is well suited for task-based approaches.

In the current version of this whitepaper, we discuss the project phases, briefly outline the benchmark problem, and discuss certain implementation challenges, the input dataset, as well as the desired output format and verification checks each implementation needs to pass.

Revision History

Revision	Date	Author(s)	Description
v1.0	05/26/2026	Gregor Daiss	Phase 1: Initial whitepaper draft

Contents

1 Introduction **3**

2 Project Phases **4**

3 Benchmark Problem Overview **5**
3.1 N-Body using the Brute-Force Algorithm 5
3.2 N-Body using the Barnes-Hut Algorithm 6

4 Implementation Specifications **6**
4.1 Comparable MPI/OpenMP Implementation (PEPC) 7
4.2 Requirements for our Implementations (Discussion) 7

5 Benchmark Input Problems **9**

6 Correctness Verification and Output Format **9**

7 Targeted Hardware Platforms **9**

8 Potential Future Work **10**

1 Introduction

Our goal is to benchmark various AMT runtime system using some computational problem. When it comes to choosing that problem, it would be beneficial to pick one:

1. That requires some irregular graph/tree structure and can reasonably be expected to benefit from a task-based approach.
2. That is a well-known problem with many existing implementations that use the classical HPC tools (MPI/OpenMP).

This would make our AMT system benchmark both more interesting and more approachable for a wider audience beyond the current AMT community (compared to using a more simple, regular problem or a completely synthetic problem).

Barnes-Hut as a Benchmark Problem: A good candidate for such a well-known problem are gravitational nbody simulation with Barnes Hut Algorithm. There are several reasons for using this specific algorithm as a benchmark.

1. For a tree-based algorithm, it is still simple and very well-known. In fact, it is known to be used in HPC lectures and tutorials aiming at students (for example, see [5]).
2. As such, it is an easy benchmark application to understand and there are existing, scalable implementations to compare to [7].
3. Yet, the algorithm requires an irregular tree-structure and creating efficient distributed implementation poses several challenges, ranging from the distributed tree traversals to load balancing issues to inefficient compute kernels due to work starvation.
4. In-between its irregular structure and these challenges, this is also a benchmark application that seem to fit the scope of task-based programming and AMT systems very well (see [9], [2] and [3] for earlier, related implementations).

In short, a nbody simulation with Barnes Hut would be a benchmark application that is a) reasonably easy to understand, b) an benchmark where AMT systems can provide a real benefit and c) an application with many existing MPI/OpenMP implementation to compare to. Hence, it would check our benchmark requirements.

Challenges: However, using Barnes-Hut as a benchmark application also comes with some challenges for ourselves. The primary downside is that a Barnes-Hut implementation can become quite complex once one moves to a distributed implementation and applies optimizations. This can make comparing implementations with different AMT systems challenging as performance numbers and code metrics will depend on how well optimized an implementation is.

Whitepaper Purpose: To mitigate the aforementioned challenges, the primary purpose of this document is to discuss implementation choices and optimization options in advance to streamline our implementations of the Barnes-Hut benchmark as best as possible. As such, the first draft of this whitepaper is not intended to provide a comprehensive implementation guide just yet, instead it is meant to spark an ongoing discussing about

the various implementation and benchmarking aspects as they are likely to evolve as soon as each team starts to implement to benchmark.
Only the final version of this whitepaper is meant to set and explicit guideline regarding the implementation and the benchmark problem itself. Adapting each implementation to the specifications of the final whitepaper version allows us to perform meaningful comparisons between the various AMT implementation. Furthermore, it makes our eventual comparison more accessible as people can look up the specifications. This should also ease future work.

Structure: We first discuss the outline of the project and its phases in Section 2, before describing the benchmark problem itself in Section 3. In Section 4 take a look at an existing MPI/OpenMP implementation of the Barnes-Hut algorithm in order to get an idea about likely implementation challenges and how they have been approached by existing solutions. In the Sections after that, we briefly cover the input dataset generator, the output format and verification tests and the target machine. We conclude with a brief outlook what future work could also be enabled by having these benchmarks available for the respective AMT systems.

2 Project Phases

It is sensible to split the project into multiple phases, particularly when it comes to the specification of the benchmark problem and the implementation choices to allow room for discussion.

- Phase 1: (Current). Starts with the initial whitepaper draft. Each team should start planning how to implement the Barnes-Hut algorithm in their respective AMT and provide feedback on the given choices regarding the implementation. Crucially, if relevant implementation aspects are not covered in the initial whitepaper these should be contributed.

- Phase 2: Using the feedback (and discussions) we finalize the whitepaper to provide a definitive guideline what each implementation should entail (and which simplications/optimization are permissible). This whitepaper version should also include an agreed upon benchmark scenario as well as correctness verifications that each implementation has to pass. It should also include an agreed upon output format and an intended target machine architecture.

- Phase 3: Each team adapts their implementation prototype to the specifications of the final whitepaper and makes sure it can run the benchmark scenario on the targeted machine (or a machine similar to it). Optimization not specific to the AMTs should be shared if possible/sensible.

- Phase 4: We run all benchmarks on the target machine. This should happen no later than January 2027.

- Phase 5: Writeup for SC27.

3 Benchmark Problem Overview

3.1 N-Body using the Brute-Force Algorithm

Each body i has a mass m_i, a position $\mathbf{p}_i$, a velocity $\mathbf{v}_i$ and an acceleration $\mathbf{a}_i$, with $\mathbf{r}_{ij} = \mathbf{p}_j - \mathbf{p}_i$ and $||\mathbf{r}_{ij}||_2$ being the Euclidean distance between $\mathbf{p}_i$ and $\mathbf{p}_j$.
The gravitational force between two bodies is given by

$$\mathbf{f}_{ij} = G\frac{m_j m_i}{||\mathbf{r}_{ij}||_2^2} \cdot \frac{\mathbf{r}_{ij}}{||\mathbf{r}_{ij}||_2} \tag{1}$$

. Thus, the gravitional force upon on body i is given by the sum of all these forces as such:

$$\mathbf{F}_i = G \cdot m_i \cdot \sum_{j=0, i \neq j}^{N} \frac{m_j \mathbf{r}_{ij}}{||\mathbf{r}_{ij}||_2^3} \tag{2}$$

. Given $F_i = m_i a_i$ we could already determine a_i, however we modify the equation to determine a_i by using a softening factor ϵ^2 which limits the magnitude of the force between bodies. This is important as our model does not take collisions into account, hence the forces could become arbitrarly large otherwise.
Thus the acceleration of a body a_i is given by:

$$\mathbf{a}_i \approx G \cdot \sum_{j=0}^{N} \frac{m_j \mathbf{r}_{ij}}{(||\mathbf{r}_{ij}||_2^2 + \epsilon^2)^{\frac{3}{2}}} \tag{3}$$

Given the usual Newton equations of motion

$$\frac{dp_i}{dt} = v_i \tag{4}$$

$$\frac{dv_i}{dt} = a_i \tag{5}$$

we can use the the computed value for $\mathbf{a}$ to compute both the velocity and the position of time using an integration scheme such as the symplectic Euler or the leapfrog algorithm. In our case, we use the leapfrog algorithm. A good introduction to the leapfrog algorithm can be found in [8].
In a nutshell, given positions, velocities and accelerations of each body for a timestep k, when can update the positions and velocities for timestep $k+1$ as follows (the accelerations can be computed using the positions for example as in Equation 3).

$$\mathbf{v}^{k+\frac{1}{2}} = \mathbf{v}^k + \mathbf{a}^k \frac{\triangle t}{2} \tag{6}$$

$$\mathbf{p}^{k+\frac{1}{2}} = \mathbf{p}^k + \mathbf{v}^{k+\frac{1}{2}} \frac{\triangle t}{2} \tag{7}$$

$$\mathbf{p}^{k+1} = \mathbf{p}^{k+\frac{1}{2}} + \mathbf{v}^{k+\frac{1}{2}} \frac{\triangle t}{2} \tag{8}$$

$$\mathbf{v}^{k+1} = \mathbf{v}^{k+\frac{1}{2}} + \mathbf{a}^{k+1} \frac{\triangle t}{2} \tag{9}$$

For simplicity we use a fixed timestep size $\triangle t$ which the user should be able to define through a command-line parameter. Of course, using Equation 3 results in a $\mathcal{O}(n^2)$ runtime complexity for updating the accelerations of each body in a single time-step.

3.2 N-Body using the Barnes-Hut Algorithm

To achieve a better runtime complexity when computing the accelerations, we can turn to tree-based algorithms. For our purposes, we specifically turn to the Barnes-Hut algorithm [1].
Here, the key idea is that far-away bodies are less relevant for the computations of the gravitational acceleration than close-by bodies. Hence, if cluster of bodies is sufficiently far away, we can approximate them as a single body. Usually, this is done through an adaptive tree-structure (for example an Oct-Tree for three dimensional problems) which is used to partition the space. Each leaf node of this tree contains one body (though there exist implementation that allow for more bodies to better optimize the compute kernels). The non-leaf nodes contain a pseudo-body which uses the sum of the masses of all bodies in the nodes below as its own mass and their center-of-masses for its position.
Using this approximation, we do not need to visit all other bodies when calculation the gravitational acceleration for one specific body i. Instead we traverse the tree and consider all bodies in the tree-nodes we encounter during the tree-traversal (including the pseudo bodies). If the distance between the body i and a psuedo body in the current tree node is sufficiently large, we do not further traverse the children of this tree-node and instead use its pesudo body for the force calculation, approximating the influence of the all the actual bodies below this tree-node.
To define what is a suffienctly large distance we define a parameter θ. This parameter is used in an equation which defines whether a distance is sufficiently large, both based on the distance and on how large a volume a tree-node covers (by considering its edge lenght):

$$\frac{octant_edge_length}{||d||_2} < \theta \tag{10}$$

. Here $||d||_2$ is the distance between the body i and the pseudo body j. $octant_edge_length$ is the edge length of the octant defined by the tree-node of said pseudo body.
If this condition holds when considering the distance between a body i and (pseudo) body j, we use the pseudo body to approxmate the influence of all other bodies below it. If it does not hold, we instead traverse the children node of the tree-node containing the pseudo body and consider their respective bodies. If we encounter a leaf node, we directly calculate their interactions with the body i.
The large the value of θ, the less bodies are directly considered, providing a tradeoff of performance and accurancy. For $\theta = 0$, the algorithm degenerates into the brute force variant again as all bodies are considered.
Overall, given a sufficiently large θ this algorithm reduces the runtime to calculate the acceleration of one body from $\mathcal{O}(n)$ to $\mathcal{O}(logn)$ as we do not visit all tree-nodes and can thus skip a lot of the overall bodies. Hence, the overall complexity to calculate the acceleration of all bodies using the Barnes-Hut algorithm is thus $\mathcal{O}(n \cdot log(n))$.

4 Implementation Specifications

There are three key steps for a Barnes-Hut implementation

1. Tree construction

2. Tree traversals for computing the accelerations

3. Updates to positions and velocities of each body

While step 3 is trivial, creating a distributed implementation for the first two steps will be challenging! As we are targeting a SC maintrack paper, the implementation with each AMT should be both scalable and fast. This means there are some implementation details to discuss in this section. For this we look at a scalable OpenMP/MPI implementation of the Barnes-Hut algorithm and its implementation choices: The Pretty Efficient Parallel Coulomb Solver (PEPC).

4.1 Comparable MPI/OpenMP Implementation (PEPC)

PEPC is an open source library[1] for NBody problems using a non-recursive implementation of the Barnes-Hut algorithm. The data is distributed across all processes (not just replicated). While PEPC supports more NBody problems than just the gravitational NBody problem we are considering, the key challenges are the same and they achieve good distributed scalability [7].
To address the tree construction, PEPC foregoes a pointer-based data-structure. Instead, PEPC uses a hashable oct-tree (HOT) as proposed by Warren and Salmon in [6]. Here, the tree is constructed based on the Morton IDs of each body. Crucially, the construction can be efficiently parallelized. It also provides a space-filling curve which can be used to determine which tree nodes should be processed by each compute node.
Implementing this a data-structure is not simple as it requires multiple steps: converting the positions of the bodies into Morton IDs, sorting the IDs and creating the tree based on the common prefixes. Yet, in return it offers a scalable tree creation with good locality (the bodies handled by each process are close to each other). Hence, using this data-structure also helps PEPC with the second step, the calculation of the accelerations.

4.2 Requirements for our Implementations (Discussion)

For Phase 1 of this project, the exact requirements in this section are still up for discussion.
There are multiple implementation details that are important to discuss:

- How do we construct the tree and what form should it have (pointer-based or hashable oct-tree?)
- Data distribution (distribute data to compute nodes or replicate it on each node?)
- Computational efficiency (configurable number of body per tree-node allowing for optimizations?)
- Input / output / hardware requirements

We will examine and discuss these implementation choices/details in this section.

[1] Overview and code are available at `https://www.fz-juelich.de/en/jsc/about-us/structure/simulation-and-data-labs/sdl-plasma-physics/pepc`

However, for discussing them, there are three key aspects we need to keep in mind given the goals our project:

- Our AMT implementations of the benchmark should be comparable. Hence, ideally we should all align our choices regarding the key implementation details (tree construction, data distribution, data-structure optimization for computational efficiency).
- Given that we target a SC main track paper, we should aim to achieve comparable (ideally better) performance than existing MPI implementations such as PEPC. One way to do so is by picking similar implementation details.
- Ideally, we demonstrate that creating distributing implementations of tree-based problems is eased by leveraging AMTs given their task-based and distributed features. Note: this goal can be at odds with the advice mentioned just now of choosing all the same implementation details as the MPI-based codes. This conflict makes the discussion the implementation details necessary.

With these aspects in mind, let us examine the implementation details/choices in more detail now:

- **Tree Construction / Data distribution:** The tree construction is the most important thing for us to consider as it will also dictate what kind of tree we should use.

  The first option here is that we use a hashable Oct-Tree similar to PEPC instead of a pointer-based oct-tree. Hence, for this option we use the morton IDs to split the bodies according to the space-filling curve to and put them onto the respective compute nodes. The tree will be implicitly constructed from these morton IDs. Bodies can be rebalanced during subsequent tree constructions as necessary. The advantages of this option here are that it keeps us very comparable to PEPC and its proven to scale. The disadvantage is that the implementation is not trivial (for example, the tree construction for hashable octrees involves a distributed sort step).

  Alternatively, the second option is to go for a more traditional pointer-based distributed octree and create it top-down. The comes with the advantage of being more straight-forward to implement. However, with this approach it is unclear if we can reach a performance comparable to the ones reached the MPI-based alternatives (specifically PEPC), even given that the task-based nature of our runtimes should ease this approach somewhat. That said, if we use it and reach good/similar performance with this simpler approach it would highlight the usefulness of AMTs when it comes to distributed tree structures.

  We should discuss which of these approaches to take! However, in either case the bodies need to split across the compute nodes (as opposed to replicate all data on all compute nodes).

- **Computational efficiency:** This point is more straightforward detail to decide: The maximum number of bodies in each leaf node should be configurable at the beginning of the simulation (or at compile-time). This will allow us to improve the performance (for example it allows for simpler SIMD usage) allowing us to reach the performance achieved by comparable MPI-based implementations. It also allows

us to easily conduct experiments with different task sizes and paves the way for potential future work including GPUs. When applicable, optimizations (such as SIMD usage) should be shared across our individual implementations.

- **Other Requirements:** The other details are also straightforward: Each implementation should be able to handle the intended input data format (see Section 5), support the Paraview output format and pass the energy verification critia (see Section 6). Each implementation needs to run on the hardware listed in Section 7.

5 Benchmark Input Problems

We can generate input problems using the data generator[2] provided by Alexander Van Craen. This data generator is capable of generating input files for simulating (synthehic) galaxy collisions. The input file is simply a csv file with the following columns:
id,mass,pos_x,pos_y,pos_z,vel_x,vel_y,vel_z
Hence, each body concists of an ID, a mass, an initial position and an initial velocity. The data generator is able to create input files with an arbitrary number of bodies which makes it a handy tool to test our distributed implementations at scale (particularly for testing the weak scalability).
A real-world alternative would have been to use NASA JPL's Small-Body Database, containing the bodies of our own solar system. However, while interesting in its own right, the number of total bodies in the database is limited (about 1.54 million bodies), making it less useful for larger, distributed runs.

6 Correctness Verification and Output Format

Throughout the simulation, each implementation should track the energies of the nbody simulation. Particularly, we need to track the kinetic energy E_K and the potential energy E_P:

$$E_K = \sum_{i}^{N} \frac{1}{2} m_i ||\mathbf{v}_i||_2^2 \tag{11}$$

$$E_P = -G \cdot \sum_{i<j} \frac{m_i m_j}{||\mathbf{p}_j - \mathbf{p}_i||_2} \tag{12}$$

In a stable simulation, the sum of these energies $E_T = E_K + E_P$ should stay constant. This should be tracked for verification purposes.
Also each simulation should be capable to output files for Paraview for further, manual verification of the results. It should be possible to disable the output for benchmarking.

7 Targeted Hardware Platforms

We have no specific supercomputer in mind for a target right now. However, we are aiming for a CPU-only supercomputer with the x86_64 architecture and at least AVX2 support.

[2] Available at `https://github.com/vancraar/DataGenerator`

8 Potential Future Work

Having a scalable implementation of this problem for each AMT opens up multiple avenues for future work. We can turn this into standard benchmark set for AMTs for future comparisons. Given the relative simplicity of the underlying problem, our implementations can also be used for teaching and training. Students can begin implementing the nbody problem using the normal $\mathcal{O}(n^2$ algorithm and then be guided towards a more optimized implementation in the respective AMT step by step. See [5] for an example of how nbody simulations are already used in HPC lessons.
Of course, there also several avenues to extend our benchmark implementations:

- Given the support of multiple bodies per leaf-node, we can extend our implementations eventually to also support GPU kernels making use of this feature. Here, GPU kernels can again be shared across our implementations.

- It is possible to extend the Barnes-Hut implementation to use a dual-tree algorithm instead. Here, we would not need to rewrite the entire application as the tree construction can mostly be reused, see [4] for an example of a Barnes-Hut code that was extended like this.

References

[1] Josh Barnes and Piet Hut. “A hierarchical O (N log N) force-calculation algorithm”. In: *nature* 324.6096 (1986), pp. 446–449.

[2] Hoang-Vu Dang et al. “PGAS with Lightweight Threads and the Barnes-Hut Algorithm”. In: ().

[3] Laxmikant V Kale et al. “Charm++ for productivity and performance”. In: (2012).

[4] Benedikt Steinbusch et al. “A Massively Parallel Barnes-Hut Tree Code with Dual Tree Traversal.” In: *PARCO*. 2015, pp. 439–448.

[5] Alexander Van Craen, Marcel Breyer, and Dirk Pflüger. “Introduction to Parallel and Distributed Programming using N-Body Simulations”. In: *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2024, pp. 347–354. DOI: 10.1109/SCW63240.2024.00052.

[6] Michael S Warren and John K Salmon. “A parallel hashed oct-tree n-body algorithm”. In: *Proceedings of the 1993 ACM/IEEE conference on Supercomputing*. 1993, pp. 12–21.

[7] Mathias Winkel et al. “A massively parallel, multi-disciplinary Barnes–Hut tree code for extreme-scale N-body simulations”. In: *Computer Physics Communications* 183.4 (2012), pp. 880–889. ISSN: 0010-4655. DOI: https://doi.org/10.1016/j.cpc.2011.12.013. URL: https://www.sciencedirect.com/science/article/pii/S0010465511004012.

[8] Peter Young. “Physics 115/242 The leapfrog method and other “symplectic” algorithms for integrating Newton’s laws of motion”. In: *línea]. Disponible en: http://physics.ucsc. edu/˜ peter/242/leapfrog. pdf* (2013).

[9] Junchao Zhang, Babak Behzad, and Marc Snir. “Design of a Multithreaded Barnes-Hut Algorithm for Multicore Clusters”. In: *IEEE Transactions on Parallel and Distributed Systems* 26.7 (2015), pp. 1861–1873.